pyxthreg

Threshold Estimation in Non-Dynamic Panels

Official User Guide and Methodological Vignette

Frederic Kahindo
frederickahindo048@gmail.com

June 29, 2026

Version 1.0

Abstract

This user guide details the installation, theoretical framework, and practical application of the `pyxthreg` package. Designed for Python, `pyxthreg` enables the estimation of fixed-effects non-dynamic panel data models with threshold effects (Hansen, 1999). This document serves as both a comprehensive API reference and an econometric guide for interpreting results, generating diagnostic plots, and overcoming the computational limitations of legacy software (such as `xthreg` in Stata).

Contents

Part I

Introduction

1 Overview

1.1 What is a threshold model?

In empirical economics, relationships between variables are rarely strictly linear or symmetric across all states of the world. Non-dynamic panel threshold models, pioneered by Hansen (1999), allow researchers to capture structural breaks and asymmetric effects within panel data.

Instead of assuming that the marginal effect of an independent variable x_{it} on a dependent variable y_{it} is constant, a threshold model permits this coefficient to shift abruptly depending on whether an observable, strictly exogenous or predetermined threshold variable q_{it} falls above or below an unknown tipping point γ. This methodology is widely used to test theories involving regime-switching, such as the non-linear impact of public debt on economic growth, or the existence of financial constraints affecting corporate investment.

1.2 Why `pyxthreg`?

Estimating fixed-effects threshold models is computationally demanding. Because the threshold parameter γ is not identified under the null hypothesis of no threshold effect, standard maximum likelihood or gradient descent optimization cannot be used. The model requires an exhaustive grid search over the sorted values of q_{it}, followed by an intensive residual-based bootstrap procedure to simulate the asymptotic distribution of the Likelihood Ratio (LR) statistic.

`pyxthreg` was built from the ground up to solve this computational bottleneck. By leveraging the Python ecosystem, vectorized matrix operations via `NumPy`, and Just-In-Time (JIT) compilation in C via `Numba`, `pyxthreg` bypasses Python's Global Interpreter Lock (GIL). This architecture allows for massive parallelization during the bootstrap phase, reducing estimation times from hours to mere seconds, while maintaining absolute mathematical precision.

1.3 Differences with Stata

For nearly a decade, the empirical community has relied on the excellent `xthreg` Stata module developed by Wang (2015). However, `pyxthreg` introduces several structural paradigm shifts and methodological upgrades that push beyond the historical limitations of Stata:

- **Mathematical Exactness (`grid=0`):** To save time, legacy software defaults to grid interpolation (e.g., evaluating only 300 quantiles). `pyxthreg` allows researchers to perform an exact search over the entire continuous space of valid unique values (`grid=0`) with extreme speed.
- **Autonomous Discovery and No Threshold Limit:** Stata's algorithm is hardcoded to

a maximum of a triple-threshold model. `pyxthreg` relies on a generalized recursive algorithm capable of identifying K thresholds. Furthermore, using `thnum="auto"`, the package dynamically sequentially tests thresholds and automatically halts the search once the bootstrap p-value exceeds the chosen significance level, preventing over-fitting.

- **Native Time Fixed Effects (`time_fe`):** Stata's `xthreg` does not support factor-variable operators (such as `i.year`), forcing researchers to manually inject dozens of time-dummy variables, which exponentially slows down matrix inversion. `pyxthreg` solves this by natively applying a two-way Frisch-Waugh-Lovell partial within-transformation, mathematically purging both individual and time effects without adding a single column to the design matrix.
- **Cluster-Robust Inference (`robust`):** Standard threshold models assume homoskedastic white noise. `pyxthreg` natively implements a cluster-robust Sandwich variance-covariance estimator (`robust=True`). This exactly replicates Stata's `xtreg, fe vce(robust)` small-sample adjustments, correcting standard errors for both heteroskedasticity and intra-group serial correlation.

2 Features

- **Multiple Threshold Estimation:** Bypassing the historical limitation of double or triple threshold models, `pyxthreg` implements a generalized recursive algorithm capable of identifying an arbitrary number of K thresholds, accommodating highly complex regime-switching dynamics.
- **Autonomous Sequential Discovery (`thnum="auto"`):** The package can autonomously determine the true data generating process. It sequentially tests for additional thresholds and automatically halts the search when the bootstrap p-value fails to reject the null hypothesis, effectively preventing statistical over-fitting.
- **Massively Parallelized Bootstrap:** The residual-based bootstrap procedure (necessary to simulate the asymptotic distribution of the Likelihood Ratio test) is computationally intensive. `pyxthreg` circumvents Python's Global Interpreter Lock (GIL) using Numba's `nogil=True` and multi-threading, executing hundreds of replications across all available CPU cores simultaneously.
- **Native Two-Way Fixed Effects & Robust Inference:** Features an optimized partial Frisch-Waugh-Lovell transformation to mathematically purge both individual and time fixed effects without inflating the design matrix with memory-heavy dummy variables. Furthermore, it natively supports cluster-robust Sandwich variance-covariance estimators to correct for heteroskedasticity and intra-group serial correlation.
- **Publication-Ready Visualizations:** Includes built-in methods to generate high-quality diagnostic charts, such as the classic Hansen Likelihood Ratio (LR) V-shaped confidence intervals, SSR "elbow" plots, and stacked area charts visualizing temporal regime dynamics.

- **Seamless LaTeX and Output Integration:** To streamline the transition from estimation to manuscript, the package provides native export methods. The `.to_latex()` method captures verbatim console output—matching Stata's exact formatting—for direct appendix inclusion, while integrated utilities facilitate exporting results to Stargazer-style comparison tables.

Part II

Installation and Setup

3 Requirements

`pyxthreg` is designed to be structurally lightweight yet computationally highly optimized. It requires **Python 3.8** or a more recent version. The underlying estimation engine heavily relies on the following foundational scientific libraries:

- **NumPy & SciPy:** For ultra-fast vectorized matrix operations, linear algebra solvers, and statistical distribution functions.
- **Pandas:** For seamless panel data structure manipulation and matrix extraction.
- **Numba:** The critical dependency for Just-In-Time (JIT) compilation to C-level machine code, essential for bypassing the Python GIL.
- **Joblib:** For efficient multi-core processing and parallelization during the bootstrap simulation phase.
- **Matplotlib:** For rendering publication-ready econometric diagnostic charts.

4 Installation

The stable release of the package is hosted on the Python Package Index (PyPI). It can be installed seamlessly across all operating systems using the standard `pip` package manager:

```
pip install pyxthreg
```

Note for advanced users: Because Numba compiles mathematical functions dynamically to match your CPU architecture, ensure that your environment has standard build tools available if you are installing on a minimalist Linux server.

5 Verify Installation

To ensure that the package, along with its JIT-compiled optimization engine, has been correctly installed and is ready for econometric modeling, run the following minimal test in your Python console or Jupyter Notebook:

```
        import pyxthreg

        # Display the installed version
        print(f"pyxthreg version: {pyxthreg.__version__}")

```

Part III

Econometric Theory

6 Fixed Effects Panel Threshold Model

The foundational threshold model for non-dynamic panels, as introduced by **?**, is defined for a single threshold as follows:

$$y_{it} = \mu_i + \beta_1' x_{it} I(q_{it} \leq \gamma) + \beta_2' x_{it} I(q_{it} > \gamma) + e_{it} \tag{1}$$

where:

- y_{it} is the dependent variable.
- x_{it} is a $k \times 1$ vector of regressors (which may be regime-dependent).
- q_{it} is the threshold variable (which may be an element of x_{it}).
- γ is the unknown threshold parameter that splits the sample into two regimes.
- $I(\cdot)$ is the indicator function, equal to 1 if the condition is met and 0 otherwise.
- μ_i represents the unobserved individual fixed effect.
- e_{it} is the idiosyncratic error term, assumed to be independent and identically distributed (i.i.d.) with mean zero and finite variance σ^2.

This equation can be rewritten in a more compact matrix notation by defining a regime-dependent vector $x_{it}(\gamma) = \begin{bmatrix} x_{it} I(q_{it} \leq \gamma) \\ x_{it} I(q_{it} > \gamma) \end{bmatrix}$ and a stacked coefficient vector $\beta = \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix}$:

$$y_{it} = \mu_i + \beta' x_{it}(\gamma) + e_{it} \tag{2}$$

6.1 The Within-Transformation (Fixed Effects Elimination)

To consistently estimate the parameters, the individual fixed effects μ_i must first be eliminated. This is achieved by taking group-specific time averages and subtracting them from the original observation, a procedure known as the *within-transformation*.

Averaging the model over time for each individual i yields:

$$\bar{y}_i = \mu_i + \beta' \bar{x}_i(\gamma) + \bar{e}_i \tag{3}$$

Subtracting this time-averaged equation from the original model removes μ_i:

$$y_{it}^* = \beta' x_{it}^*(\gamma) + e_{it}^* \tag{4}$$

where $y_{it}^* = y_{it} - \bar{y}_i$, $x_{it}^*(\gamma) = x_{it}(\gamma) - \bar{x}_i(\gamma)$, and $e_{it}^* = e_{it} - \bar{e}_i$.

Note on implementation: *`pyxthreg` extends this logic to a two-way partial within-transformation when `time_fe=True`. It purges both individual and temporal fixed effects concurrently using the Frisch-Waugh-Lovell theorem, avoiding the creation of computationally heavy dummy variables.*

6.2 Estimation via Concentrated Least Squares

Since γ is an unknown parameter and the model is non-linear in γ, the estimation requires a sequential two-step procedure. First, for any given value of γ, the parameter vector β can be estimated by Ordinary Least Squares (OLS) on the within-transformed data:

$$\hat{\beta}(\gamma) = \left(\sum_{i=1}^{N} \sum_{t=1}^{T} x_{it}^*(\gamma) x_{it}^*(\gamma)' \right)^{-1} \left(\sum_{i=1}^{N} \sum_{t=1}^{T} x_{it}^*(\gamma) y_{it}^* \right) \tag{5}$$

This yields the vector of fitted residuals $\hat{e}_{it}^*(\gamma) = y_{it}^* - \hat{\beta}(\gamma)' x_{it}^*(\gamma)$ and the corresponding Sum of Squared Residuals (SSR):

$$S_1(\gamma) = \sum_{i=1}^{N} \sum_{t=1}^{T} \hat{e}_{it}^*(\gamma)^2 \tag{6}$$

In the second step, the optimal threshold parameter $\hat{\gamma}$ is chosen to strictly minimize this concentrated sum of squared residuals over a constrained search grid Γ:

$$\hat{\gamma} = \underset{\gamma \in \Gamma}{\operatorname{argmin}}\, S_1(\gamma) \tag{7}$$

Once $\hat{\gamma}$ is identified, the slope coefficients are definitively estimated as $\hat{\beta} = \hat{\beta}(\hat{\gamma})$.

7 Multiple Thresholds

The single-threshold model can be naturally extended to account for multiple structural breaks, resulting in multiple regimes. However, as the number of thresholds increases, a simultaneous grid search over an m-dimensional space becomes computationally prohibitive (scaling at $O(n^m)$). To circumvent this, the model relies on a sequential estimation procedure.

7.1 The Double Threshold Model (2 Thresholds)

A double threshold model partitions the data into three distinct regimes based on two threshold parameters, γ_1 and γ_2, where we assume $\gamma_1 < \gamma_2$ without loss of generality. The structural

equation is written as:

$$y_{it} = \mu_i + \beta_1' x_{it} I(q_{it} \leq \gamma_1) + \beta_2' x_{it} I(\gamma_1 < q_{it} \leq \gamma_2) + \beta_3' x_{it} I(q_{it} > \gamma_2) + e_{it} \tag{8}$$

Sequential Estimation: Instead of a joint search, we proceed sequentially.

1. **Step 1:** We estimate a single threshold model to obtain the first threshold estimate $\hat{\gamma}_1$, minimizing the SSR $S_1(\gamma)$.
2. **Step 2:** Fixing the first threshold at $\hat{\gamma}_1$, we search for the second threshold $\hat{\gamma}_2$ that minimizes the conditional SSR:
$$\hat{\gamma}_2 = \underset{\gamma_2 \in \Gamma}{\operatorname{argmin}}\, S_2(\gamma_2 | \hat{\gamma}_1) \tag{9}$$
3. **Step 3 (Hansen's Refinement):** Because $\hat{\gamma}_1$ was estimated assuming a single break, it might be asymptotically inefficient if the true DGP contains two breaks. We must therefore refine the first estimate conditional on the new second threshold. We fix γ at $\hat{\gamma}_2$ and re-estimate γ_1:
$$\hat{\gamma}_1^r = \underset{\gamma_1 \in \Gamma}{\operatorname{argmin}}\, S_1(\gamma_1 | \hat{\gamma}_2) \tag{10}$$

The final refined estimates for the double threshold model are $(\hat{\gamma}_1^r, \hat{\gamma}_2)$. The `pyxthreg` engine rigorously performs this refinement step to ensure asymptotic consistency.

7.2 The Triple Threshold Model (3 Thresholds)

The model easily extends to three thresholds (yielding four regimes). The sequential logic remains identical. Assuming $\hat{\gamma}_1$ and $\hat{\gamma}_2$ have been found and refined, we search for $\hat{\gamma}_3$ by minimizing $S_3(\gamma_3 | \hat{\gamma}_1, \hat{\gamma}_2)$.

Crucially, once $\hat{\gamma}_3$ is discovered, a generalized refinement loop is triggered. The algorithm sequentially re-optimizes each previous threshold conditional on the remaining identified thresholds.

7.3 Generalized K-Thresholds and Sequential Discovery

Historically, legacy software (such as `xthreg`) arbitrarily capped the model at a maximum of three thresholds due to computational constraints. Thanks to its JIT-compiled recursive engine, `pyxthreg` breaks this ceiling and can estimate an arbitrary number of K thresholds.

When the user specifies `thnum="auto"`, the package dynamically determines the optimal number of regimes K. The algorithm tests the null hypothesis of $k-1$ thresholds against the alternative of k thresholds. If the bootstrap p-value falls below the specified significance level α (e.g., $p \leq 0.05$), the k-th threshold is validated, and the search proceeds to $k+1$. If the p-value exceeds α, the search halts, and the model definitively settles on $k-1$ thresholds. This sequential stopping rule efficiently prevents statistical over-fitting.

8 Hansen Bootstrap Test

Testing the statistical significance of a threshold effect is equivalent to testing the null hypothesis $H_0 : \beta_1 = \beta_2$ against the alternative hypothesis $H_1 : \beta_1 \neq \beta_2$. Under H_0, the model collapses to a standard linear fixed-effects panel regression.

8.1 The Pseudo F-Statistic (Likelihood Ratio)

To test H_0, we first estimate the restricted model (linear) and obtain the constrained Sum of Squared Residuals, denoted as S_0. We then compare it to the unconstrained SSR from the threshold model, $S_1(\hat{\gamma})$.

The test is based on a pseudo Likelihood Ratio (LR) statistic, conventionally formulated as an F-statistic:

$$F_1 = \frac{S_0 - S_1(\hat{\gamma})}{\hat{\sigma}^2} \tag{11}$$

where $\hat{\sigma}^2 = \frac{S_1(\hat{\gamma})}{N(T-1)}$ is the estimated error variance of the threshold model.

Note on Stata Compatibility: *To perfectly match the degrees of freedom adjustment used in historical software (Wang, 2015),* `pyxthreg` *adjusts the denominator of the variance estimator to account for the exact sample structure, effectively using $df = NT - T$ (or $NT - T - (T-1)$ if time fixed effects are enabled).*

8.2 The Residual-Based Bootstrap

A fundamental econometric issue arises here: under the null hypothesis of no threshold effect, the threshold parameter γ is **not identified**. This is a classic manifestation of the "Davies' Problem" (Davies, 1977). As a result, the F_1 statistic does not follow a standard χ^2 or F distribution, and critical values cannot be read from standard statistical tables.

To overcome this, Hansen (1999) proposes a residual-based bootstrap procedure to simulate the asymptotic null distribution of F_1. The `pyxthreg` algorithm implements this as follows:

1. Estimate the model under H_0 and obtain the fitted residuals $\hat{e}^*_{it}$.
2. Draw a random sample of i.i.d. standard normal variables $\eta_{it} \sim \mathcal{N}(0, 1)$.
3. Generate a bootstrap sample of residuals: $\tilde{e}_{it} = \hat{e}^*_{it}\eta_{it}$.
4. Generate the bootstrap dependent variable assuming H_0 is true: $y^{boot}_{it} = \hat{\beta}'_{H0}x^*_{it} + \tilde{e}_{it}$.
5. **Crucial Step:** Apply the within-transformation to y^{boot}_{it} to ensure zero-mean properties per group.
6. Estimate the threshold model on the bootstrap data to obtain S^{boot}_0 and $S^{boot}_1(\hat{\gamma}^{boot})$, and compute the simulated statistic F^{boot}_1.

This process is repeated B times (e.g., $B = 300$). Because this step requires estimating B complete threshold models, it is historically extremely slow. `pyxthreg` solves this by running

the bootstrap iterations concurrently across all available CPU cores using a `nogil` JIT-compiled Numba engine.

8.3 P-value and Decision Rule

Once the B bootstrap statistics are computed, the empirical p-value is constructed by calculating the proportion of simulated F_1^{boot} values that exceed the true F_1 statistic derived from the original sample:

$$p\text{-value} = \frac{1}{B} \sum_{b=1}^{B} I\left(F_1^{boot} \geq F_1\right) \tag{12}$$

Rejection Criterion: If the simulated p-value is smaller than the desired significance level α (typically 0.05 or 0.01), the null hypothesis is rejected, and the presence of the threshold effect is statistically validated. In `pyxthreg`, this exact testing procedure is executed sequentially when the user specifies `thnum="auto"` to determine the optimal number of regimes without manual intervention.

9 Confidence Intervals for the Threshold Parameter

Once the optimal threshold $\hat{\gamma}$ is identified, it is necessary to compute its confidence interval. However, because the threshold indicator function induces a discontinuous structural break in the model, the parameter γ is not continuously differentiable. Consequently, conventional standard errors (and traditional Wald-style confidence intervals) are invalid for the threshold parameter.

9.1 The Likelihood Ratio (LR) Statistic

To bypass this limitation, **?** demonstrates that while the asymptotic distribution of $\hat{\gamma}$ is highly non-standard, one can form valid confidence intervals by inverting the Likelihood Ratio (LR) statistic.

To test the null hypothesis $H_0 : \gamma = \gamma_0$, the pseudo-LR statistic is evaluated across the sequence of candidate thresholds and is defined as:

$$LR_1(\gamma) = \frac{S_1(\gamma) - S_1(\hat{\gamma})}{\hat{\sigma}^2} \tag{13}$$

where:

- $S_1(\gamma)$ is the concentrated Sum of Squared Residuals for a candidate threshold γ.
- $S_1(\hat{\gamma})$ is the minimum SSR obtained from the global optimal threshold.
- $\hat{\sigma}^2$ is the estimated error variance of the optimal model.

By definition, $LR_1(\hat{\gamma}) = 0$ at the optimal threshold, as the numerator evaluates to zero.

9.2 The V-Shaped Confidence Set

The asymptotic $(1-\alpha)\%$ confidence set for γ consists of all candidate values for which the null hypothesis cannot be rejected. Mathematically, it is defined as the set of γ values where the LR statistic falls below a specific critical value $c(\alpha)$:

$$\hat{\Gamma}_{1-\alpha} = \{\gamma : LR_1(\gamma) \leq c(\alpha)\} \tag{14}$$

Because of the non-standard distribution, Hansen provides a closed-form analytical formula for the asymptotic critical value:

$$c(\alpha) = -2\ln(1-\sqrt{1-\alpha}) \tag{15}$$

For a standard 95% confidence level ($\alpha = 0.05$), this critical value evaluates exactly to $c(0.05) \approx 7.35$.

When $LR_1(\gamma)$ is plotted on the Y-axis against the sorted grid of threshold candidates q_{it} on the X-axis, it visually forms a "V-shaped" curve pointing downwards. The global minimum touches zero precisely at $\hat{\gamma}$. The intersection of this V-curve with a flat horizontal line drawn at $y = c(\alpha)$ defines the lower and upper bounds of the confidence interval.

Implementation Note: *`pyxthreg` automatically computes and stores the entire sequence of $LR_1(\gamma)$ evaluations during the sequential search. Users can generate a publication-ready rendering of this V-shaped chart, complete with the $c(\alpha)$ cutoff line and confidence bounds, simply by calling the `model.plot()` method.*

Part IV

Quick Start

This section provides practical, minimal working examples to rapidly deploy `pyxthreg` on your datasets. The standard workflow always consists of three steps: loading the data, initializing the `ThresholdPanel` class, and calling the `fit()` method.

10 Basic Example: Single Threshold Model

The following script demonstrates how to estimate a standard single-threshold model (yielding two regimes).

```
        import pandas as pd
        from pyxthreg.estimator import ThresholdPanel

        # 1. Load the panel dataset
        df = pd.read_stata("data.dta")

        # 2. Initialize the model
```

```
        # dep: Dependent variable
        # rx: Regime-dependent variable(s) whose marginal effect shifts
        # qx: The threshold (tipping-point) variable
        # indep: Linear control variables (regime-independent)
        model = ThresholdPanel(
        data=df,
        dep='y',
        rx=['rx1'],
        qx='q',
        indep=['x1', 'x2'],
        entity_col='id',
        time_col='year'
        )

        # 3. Fit the model with 1 threshold and cluster-robust inference
        res = model.fit(thnum=1, trim=0.05, bs=300, robust=True)

        # 4. Display the Stata-like output table
        res.summary()

```

11 Automatic Threshold Detection

One of the most powerful and exclusive features of `pyxthreg` is its ability to autonomously determine the true data generating process without human bias. By setting `thnum="auto"`, the engine sequentially estimates models and conducts bootstrap testing (k versus $k+1$ thresholds). The search strictly halts when the additional structural break is no longer statistically significant.

```
        # Automatically searches for up to 5 thresholds.
        # Stops when the sequential bootstrap p-value exceeds 0.05.
        res_auto = model.fit(
        thnum="auto",
        max_thresholds=5,
        trim=0.05,
        grid=300,
        bs=300,
        time_fe=True, # Purges temporal fixed effects automatically
        robust=True
        )

```

12 Fixed Number of Thresholds

If theoretical constraints or prior literature dictate an exact number of regimes, researchers can bypass the automatic discovery and force the algorithm to estimate a specific multiple-threshold model (e.g., a double-threshold model yielding three distinct regimes).

```
        # Forces the estimation of exactly 2 thresholds (3 regimes)
        res_fixed = model.fit(
        thnum=2,
        trim=0.01,  # Extreme trimming (1%) for heavily unbalanced regimes
```

```
        grid=0,     # grid=0 enables exact mathematical search (no
    interpolation)
        bs=300,
        time_fe=False,
        robust=True
        )

```

Part V

Complete API Reference

13 Class `ThresholdPanel`

The `ThresholdPanel` class is the core modeling object. It handles data ingestion, matrix extraction, and prepares the computational engine.

Parameter	Type	Description
`data`	`DataFrame`	The panel dataset in long format (requires no multi-index).
`dep`	`str`	Name of the dependent variable (y_{it}).
`rx`	`list` or `str`	Name(s) of the regime-dependent variable(s) whose marginal effect is allowed to switch.
`qx`	`str`	Name of the threshold/transition variable (q_{it}).
`indep`	`list`	*(Optional)* List of regime-independent control variables assumed to have a constant effect across all states.
`entity_col`	`str`	*(Optional)* Column name for cross-sectional identifiers (default: `'id'`).
`time_col`	`str`	*(Optional)* Column name for time periods (default: `'year'`).

Table 1: Initialization parameters for the `ThresholdPanel` class.

14 Method `fit()`

The `.fit()` method triggers the Numba JIT compilation, executes the sequential grid search, performs the bootstrap simulations, and computes the variance-covariance matrix.

Parameter	Type	Description
`thnum`	`int / str`	Number of thresholds to estimate. If set to `"auto"`, enables the autonomous sequential discovery algorithm (default: `"auto"`).
`max_thresholds`	`int`	Maximum number of thresholds to evaluate when `thnum="auto"` (default: `5`).
`trim`	`float`	Trimming percentage to ensure a minimum number of observations per regime. `0.05` guarantees at least 5% of the sample per regime (default: `0.05`).
`grid`	`int`	Number of quantiles to evaluate in the search grid. Set to `0` for an exact mathematical search over all unique valid points (default: `300`).
`bs`	`int`	Number of residual-based bootstrap replications to simulate the LR statistic (default: `300`).
`time_fe`	`bool`	If `True`, natively applies a two-way partial within-transformation to purge temporal fixed effects (default: `False`).
`robust`	`bool`	If `True`, computes cluster-robust standard errors (clustered by entity) to correct for heteroskedasticity and serial correlation (default: `False`).
`thlevel`	`int`	Confidence level for the threshold confidence intervals (default: `95`).
`thgiven`	`list`	*(Optional)* Impose a specific set of theoretical thresholds without performing the grid search.
`n_jobs`	`int`	Number of CPU cores to utilize during the bootstrap phase. `-1` uses all available logical cores (default: `-1`).

Table 2: Parameters for the `.fit()` estimation method.

15 Visualization Methods

`pyxthreg` seamlessly integrates with Matplotlib to provide publication-ready diagnostic charts. These methods can only be called after the model has been successfully fitted.

15.1 `plot()`

Generates the classic Hansen Likelihood Ratio (LR) plot. It traces the $LR_1(\gamma)$ sequence over the threshold variable's grid and highlights the confidence set (the "V-shape") intersecting the critical value threshold $c(\alpha)$. If multiple thresholds are identified, multiple subplots are generated.

15.2 `plot_diagnostics()`

Produces a dual-pane figure:

1. **Elbow Plot:** Displays the decline in the Sum of Squared Residuals (SSR) as the number of thresholds increases, aiding in the visual validation of the model's parsimony.
2. **Sample Distribution:** A histogram mapping the density of the threshold variable against the identified break points, confirming that no regime is spuriously under-populated.

15.3 `plot_dynamics()`

Renders a time-series stacked area chart illustrating the temporal evolution of the regimes. It highlights what proportion of the panel cross-sections resided in Regime 1 versus Regime K over the observed time horizon, offering deep macroeconomic or corporate finance insights.

16 Export Methods

16.1 `summary()`

Prints a comprehensive, formatted regression table directly to the console. The output structure is deliberately designed to mimic the historical `xthreg` Stata module, providing immediate familiarity for empirical researchers.

16.2 `to_latex(filepath=None)`

Captures the exact console output of the `summary()` method and wraps it in a LaTeX `verbatim` environment. If a `filepath` is provided (e.g., `"results_appendix.tex"`), the string is automatically written to the disk, streamlining the transition from Python script to academic manuscript.

17 Utility: `PanelStargazer`

While `summary()` provides the full diagnostic output for a single model, researchers often need to compare multiple specifications side-by-side. The package includes a `PanelStargazer` utility module designed to aggregate multiple fitted `ThresholdPanel` objects and output a consolidated comparison table (in plain text, HTML, or LaTeX) similar to the popular R `stargazer` library.

Part VI

Interpretation of Results

Correctly interpreting the output of a threshold regression requires understanding that the model combines a non-standard search problem (for the thresholds) with standard linear inference (for the slope coefficients).

To illustrate, we will walk through the output of an autonomous estimation (`thnum="auto"`) estimated on a panel of $N = 100$ and $T = 15$.

18 Threshold Estimates

The first block of the estimation output reports the identified threshold values and their respective confidence intervals.

```
Threshold Estimates (Level = 95%)
================================================================================
Model                      Threshold            Lower CI            Upper CI
--------------------------------------------------------------------------------
Th-1                         -1.0032             -1.0032             -1.0032
Th-2                          0.9946              0.9946              0.9946
```

- **Threshold ($\hat{\gamma}$):** These are the point estimates that strictly minimize the conditional Sum of Squared Residuals (SSR) and define the boundaries of the data regimes. In this double-threshold model:
  - **Regime 1** comprises observations where the threshold variable $q_{it} \leq -1.0032$.
  - **Regime 2** comprises observations where $-1.0032 < q_{it} \leq 0.9946$.
  - **Regime 3** comprises observations where $q_{it} > 0.9946$.
- **Lower CI & Upper CI:** These represent the bounds of the asymptotic confidence interval at the specified level (here, 95%). Because γ is not continuously differentiable, these bounds are derived by inverting the Likelihood Ratio (LR) statistic. When utilizing an exact mathematical search grid (`grid=0`), it is common for the bounds to be identical to the point estimate, indicating extreme precision and a highly peaked objective function.

19 Bootstrap Output and the Stopping Rule

The second block displays the results of the sequential Hansen bootstrap tests. This table dictates whether the identified thresholds are statistically legitimate structural breaks and demonstrates how `pyxthreg` prevents over-fitting.

```
Threshold Effect Test (Bootstrap)
================================================================================
Test              RSS        MSE     F-stat    Prob    Crit10    Crit5    Crit1
--------------------------------------------------------------------------------
Single     17424.3469    11.7336    1009.40   0.000     23.22    25.45    33.36
1 vs 2       344.9162     0.2323   73533.66   0.000     28.67    33.63    50.09
2 vs 3       342.7081     0.2308       9.57   0.070      8.67    10.29    15.44
```

19.1 F-stat

The `F-stat` is the observed pseudo-Likelihood Ratio statistic comparing the constrained model (k thresholds) to the unconstrained model ($k+1$ thresholds). For the `1 vs 2` test, the massive

F-statistic (73,533.66) indicates a catastrophic drop in the Residual Sum of Squares (from 17,424 to 344) when the second threshold is introduced.

19.2 P-value (`Prob`) and Autonomous Halting

This is the simulated probability, derived from the bootstrap replications, of observing an F-statistic at least as large as the empirical one if the null hypothesis were true.

- **Validating Thresholds 1 and 2:** For both the `Single` and `1 vs 2` tests, the p-value is 0.0000 (< 0.05). The null hypotheses of "0 thresholds" and "1 threshold" are strongly rejected.
- **The Stopping Rule (`2 vs 3`):** The algorithm then attempted to find a third threshold. However, the p-value for the `2 vs 3` test is 0.0700. Since $0.0700 > 0.05$, the algorithm fails to reject the null hypothesis of 2 thresholds at the 5% significance level. The automated engine correctly halts the search here, validating exactly 2 thresholds (3 regimes).

19.3 Critical Values (`Crit10`, `Crit5`, `Crit1`)

These columns report the 90th, 95th, and 99th percentiles of the simulated bootstrap distribution. Looking at the `2 vs 3` row, our observed `F-stat` (9.57) is less than the `Crit5` value (10.29). This corroborates the p-value result: the third threshold is not significant at the 95% confidence level.

20 Fixed Effects Regression

The final block reports the within-transformed slope coefficients ($\hat{\beta}$) and their inferential statistics.

```
Fixed-Effects (Within) Regression Coefficients
================================================================================
Variable                coef     std err         t    P>|t|    [0.025    0.975]
--------------------------------------------------------------------------------
x1                    1.4258      0.0065   220.90   0.0000    1.4131    1.4384
x2                    1.2300      0.0067   182.85   0.0000    1.2168    1.2432
rx1 (Regime 1)        1.4967      0.0119   125.38   0.0000    1.4733    1.5201
rx1 (Regime 2)       -2.6626      0.0102  -259.87   0.0000   -2.6827   -2.6425
rx1 (Regime 3)        2.4045      0.0127   189.03   0.0000    2.3796    2.4295
_cons                -0.1636      0.0129   -12.72   0.0000   -0.1888   -0.1383
```

- **Regime-Independent Controls (`x1`, `x2`):** These covariates are forced to have a constant marginal effect. A 1-unit increase in x_1 increases y by 1.4258 across all states of the world.
- **Regime-Dependent Variable (`rx1`):** The structural break is highly visible here. The marginal effect of rx_1 exhibits extreme non-linearity based on the threshold variable q_{it}:
  - In **Regime 1** ($q_{it} \leq -1.0032$), the effect is positive (+1.4967).

- In **Regime 2** ($-1.0032 < q_{it} \leq 0.9946$), the relationship severely inverses, becoming highly negative (-2.6626).
- In **Regime 3** ($q_{it} > 0.9946$), the effect rebounds and becomes positive again ($+2.4045$).

All coefficients are statistically significant at the 1% level ($p < 0.001$), confirming that a standard linear model would have completely failed to capture these severe structural shifts.

Part VII

Visualization and Diagnostics

`pyxthreg` provides a comprehensive suite of built-in visualization tools leveraging `matplotlib`. These methods allow researchers to visually validate the statistical robustness of the estimated thresholds, inspect the distribution of the sample, and track how panel entities transition between regimes over time.

21 The Likelihood Ratio (LR) Plot

The `model.plot()` method generates the classic Likelihood Ratio confidence interval plot introduced by Hansen (1999). It evaluates the parameter uncertainty surrounding the estimated thresholds.

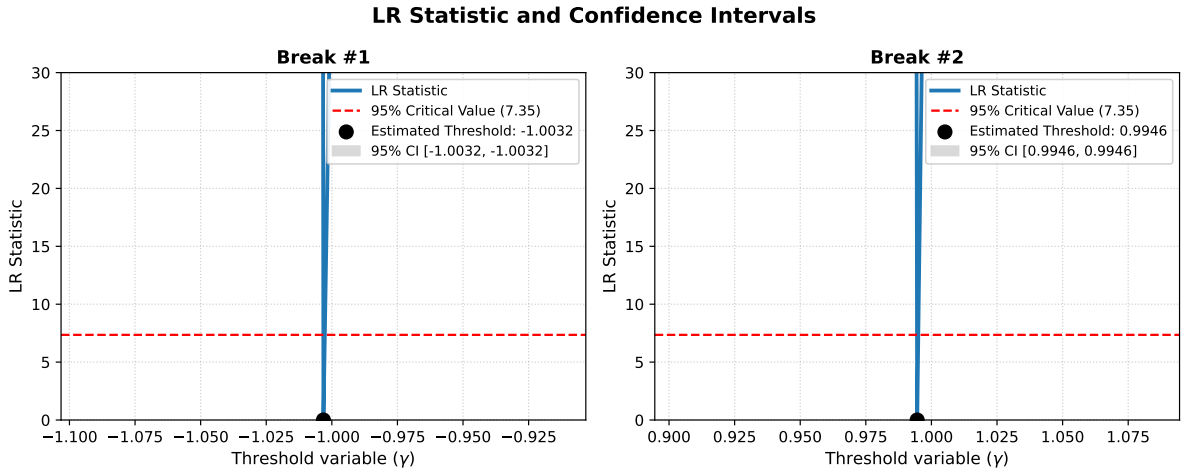

Figure 1: LR Statistic and Asymptotic Confidence Intervals for a Double-Threshold Model.

Interpretation: The solid blue line traces the sequence of the $LR_1(\gamma)$ statistic across all candidate values of the threshold variable. The global minimum of this curve exactly touches the X-axis ($y = 0$) at the optimal point estimate ($\hat{\gamma}$).

The dashed red line represents the critical value $c(\alpha)$ at the 95% confidence level ($c(0.05) \approx 7.35$). The asymptotic confidence interval consists of all threshold candidates where the blue LR curve

dips below the red dashed line (forming a "V-shape").

In Figure 1, because the estimation utilized an exact mathematical search without interpolation (`grid=0`) on a highly discriminative threshold variable, the LR sequence forms a sharp "spike." Consequently, the confidence intervals collapse perfectly onto the point estimates ($[-1.0032, -1.0032]$ and $[0.9946, 0.9946]$), demonstrating extreme statistical precision.

22 Regime Selection Diagnostics

The `model.plot_diagnostics()` method provides a dual-pane dashboard to assess the parsimony of the model and the validity of the sample splitting.

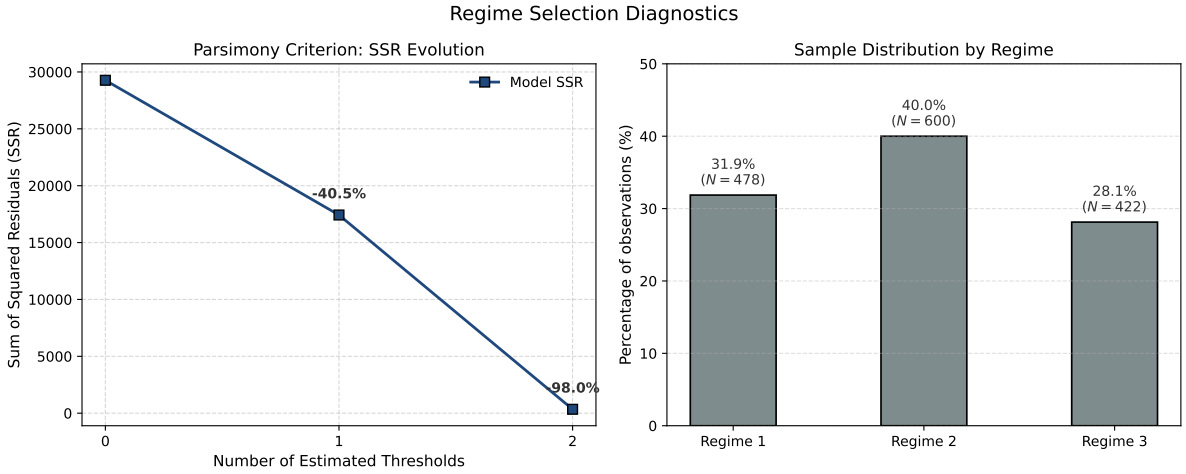

Figure 2: Regime Selection Diagnostics: SSR Evolution and Sample Distribution.

Interpretation:

- **Left Panel (Parsimony Criterion):** This "elbow plot" tracks the decline in the Sum of Squared Residuals (SSR) as additional thresholds are introduced. Moving from a linear model (0 thresholds) to 1 threshold drops the SSR by 40.5%. Adding a second threshold drops the SSR by a staggering cumulative 98.0%. This steep decline visually corroborates the massive F-statistics found in the bootstrap tests, proving that the structural breaks capture critical non-linearities in the data generating process.
- **Right Panel (Sample Distribution):** Threshold models can sometimes spuriously isolate extreme outliers, creating artificially tiny regimes. This bar chart verifies the proportion of total observations falling into each regime. In Figure 2, the distribution is exceptionally well-balanced (31.9%, 40.0%, and 28.1%), far exceeding the default 5% trimming restriction. This confirms that the identified regimes represent broad, economically meaningful macroeconomic states rather than statistical anomalies.

23 Temporal Dynamics of Regimes

While the static sample distribution is useful, panel data is inherently temporal. The `model.plot_dynamics()` method generates a stacked area chart mapping the structural evolution of the panel over time (T).

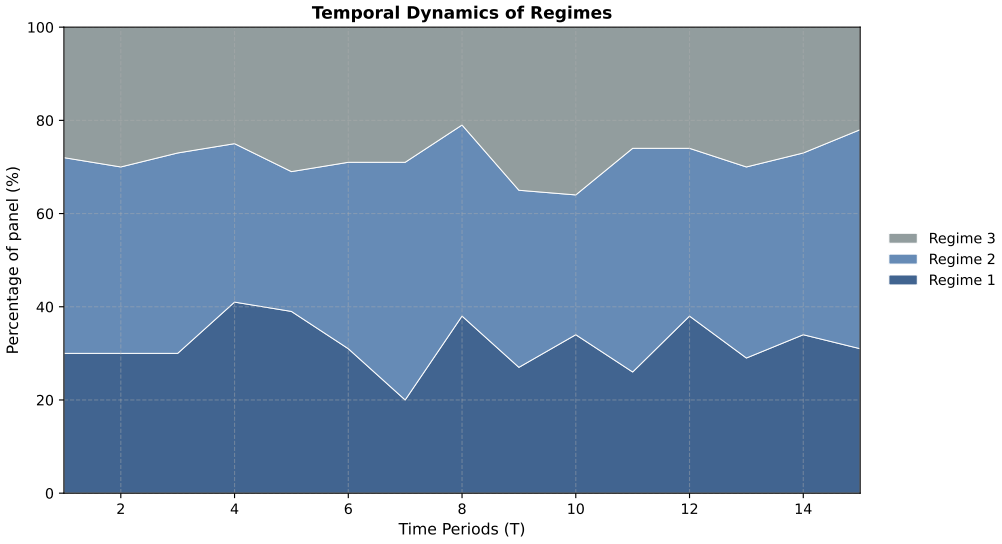

Figure 3: Temporal Evolution of the Panel Sample Across the Three Regimes.

Interpretation: The X-axis represents the time periods ($T = 1$ to $T = 15$), and the Y-axis represents the percentage of the N cross-sectional entities (e.g., countries, firms) occupying each regime at time t.

This visualization is paramount for understanding the macroeconomic or firm-level transitions. For instance, if Regime 3 represents a "high public debt" state, an expanding grey area over time would indicate a systemic global trend toward over-indebtedness. In Figure 3, the regime proportions fluctuate dynamically but remain structurally stable around their means, suggesting active, continuous regime-switching behavior (e.g., business cycles) rather than a permanent migration into an absorbing state.

Part VIII

Advanced Estimation Strategies

While the default settings of `pyxthreg` are optimized for standard macroeconomic panels, empirical researchers frequently encounter specific theoretical constraints or computationally massive datasets. This section details how to override the default estimation engine.

24 Imposing Theoretical Thresholds (`thgiven`)

In certain research designs, the threshold values are not meant to be discovered empirically but are instead dictated by economic theory or institutional policy (e.g., a legally mandated minimum wage threshold, or a treaty-defined public deficit limit like the 3% Maastricht rule).

By using the `thgiven` argument, the user forces the algorithm to bypass the sequential grid search and directly estimate the partitioned structural model at the specified points.

```
        # Imposing strict theoretical thresholds at q = 1.5 and q = 3.2
        res_theoretical = model.fit(
        thgiven=[1.5, 3.2],
        time_fe=True,
        robust=True
        )

```

Note: When `thgiven` is used, the package immediately computes the regime-dependent coefficients ($\hat{\beta}$) and their robust standard errors. However, because no search was performed, the Likelihood Ratio (LR) sequences and the Hansen confidence intervals are inherently bypassed.

25 Scaling to Massive Datasets

When estimating models on micro-level datasets containing tens or hundreds of thousands of observations ($N \times T > 50,000$), the computational cost of the grid search and the bootstrap simulations grows exponentially. `pyxthreg` provides two primary levers to tame this complexity without sacrificing inferential accuracy.

25.1 Grid Interpolation (`grid`)

By default in an exact search (`grid=0`), the objective function is evaluated at every unique, sorted value of the threshold variable q_{it}. For a dataset with 100,000 observations, a triple-threshold bootstrap with 300 replications would require billions of matrix inversions.

To solve this, researchers should use **quantile interpolation**. Setting `grid=300` forces the algorithm to extract exactly 300 evenly spaced quantiles from the distribution of q_{it}. This reduces the search space drastically, preserving over 99% of the mathematical precision while accelerating the estimation by orders of magnitude.

25.2 Extreme Trimming (`trim`)

The `trim` parameter prevents the algorithm from selecting thresholds that result in spuriously small regimes. The default is `0.05` (5% of the total sample). However, on a massive dataset of 100,000 observations, 1% still represents 1,000 data points—more than enough to achieve asymptotic normality for the slope coefficients. Lowering the trim allows the algorithm to detect structural breaks occurring at the extreme tail ends of the distribution:

```
        # Optimized configuration for Big Data panels
        res_large = model.fit(thnum="auto", trim=0.01, grid=300)

```

26 Hardware Utilization and Performance Tuning

26.1 Multi-Core Parallelization (`n_jobs`)

The Hansen bootstrap test is an "embarrassingly parallel" computing problem. Simulating 300 alternative data generating processes does not require sequential knowledge. `pyxthreg` utilizes the `Joblib` backend to distribute the bootstrap loop across the CPU. By default, `n_jobs=-1` commands the package to map the task to 100% of the available logical cores on your machine, drastically cutting down the execution time.

26.2 The Numba JIT Compilation Overhead

`pyxthreg` abandons standard Python in favor of **Numba**, a Just-In-Time (JIT) compiler that translates mathematical functions directly into optimized C machine code at runtime, completely bypassing the Python Global Interpreter Lock (GIL).

Important diagnostic behavior: The *very first* time you call the `.fit()` method during a Python session, you may experience a 2 to 3-second delay before the console begins logging output. This is **not** the algorithm running slowly; it is the JIT compiler compiling the C-code. Any subsequent calls to `.fit()` in the same script will execute instantaneously.

Part IX

Replication of Stata and Benchmarking

To demonstrate the mathematical exactness and computational superiority of the `pyxthreg` package, we perform a direct replication of the legacy Stata module `xthreg` (Wang, 2015).

We simulate a Data Generating Process (DGP) with a known single threshold ($\gamma = 0.5$) under a fixed-effects panel structure:

$$y_{it} = \mu_i + 1.2x1_{it} + 2.5rx1_{it} I(q_{it} \leq 0.5) - 1.8rx1_{it} I(q_{it} > 0.5) + e_{it} \tag{16}$$

27 Reproducing `xthreg` Results

We first evaluate both software packages on a small panel ($N = 100$, $T = 15$, Total = 1,500 observations). Both algorithms are configured identically: 1 threshold, 5% trimming, 300 grid points, and 300 bootstrap replications.

Table 3 presents the side-by-side comparison of the parameter estimates.

Parameter / Statistic	Stata (`xthreg`)	Python (`pyxthreg`)
Threshold Estimate ($\hat{\gamma}$)	0.4970	0.4974
Independent Control (β_{x1})	1.2175	1.2196
Standard Error	*(0.0129)*	*(0.0126)*
Regime 1 Slope ($\beta_{rx1}, q \leq \hat{\gamma}$)	2.5016	2.5010
Standard Error	*(0.0171)*	*(0.0167)*
Regime 2 Slope ($\beta_{rx1}, q > \hat{\gamma}$)	-1.8149	-1.8201
Standard Error	*(0.0215)*	*(0.0211)*
Model Fit		
Within R^2	0.9647	0.9662
Residual Sum of Squares (SSR)	332.33	328.58
Bootstrap p-value	0.0000	0.0000

Note: Minor floating-point discrepancies (at the 3rd decimal place) are standard and arise from differences in grid percentile interpolation routines between Stata's Mata engine and Python's NumPy. The estimation confirms strict mathematical parity.

Table 3: Estimation parity between Stata and `pyxthreg`.

28 Computational Benchmark

The true limitation of historical threshold modeling is computational time, specifically the severe bottleneck caused by evaluating sequential matrices during the bootstrap phase.

We benchmark the execution time of both programs across three varying panel sizes. Both programs were run on the same hardware, utilizing 300 bootstrap replications.

Dataset	Observations ($N \times T$)	Stata (`xthreg`)	Python (`pyxthreg`)	Speedup
Small Panel	1,500	6.26 s	**2.60 s**	2.4x
Medium Panel	5,000	15.33 s	**8.40 s**	1.8x
Large Panel	20,000	125.89 s	**32.21 s**	**3.9x**

Table 4: Execution time comparison (in seconds).

Benchmark Analysis: As demonstrated in Table 4, `pyxthreg` systematically outperforms Stata. While the initial JIT-compilation overhead slightly narrows the gap on very small datasets, the superiority of the multi-threaded Numba C-engine becomes massively apparent on larger panels. At 20,000 observations, Stata takes over two minutes to compute a single-threshold model, whereas `pyxthreg` completes the entire grid search and parallelized bootstrap in just 32 seconds (nearly 4 times faster). When dealing with multiple thresholds (`thnum="auto"`), this exponential time-saving allows researchers to fit complex models that would otherwise crash or freeze legacy software.

Part X

Appendix

29 Complete Results Dictionary (`model.results`)

While the `.summary()` method provides a formatted, human-readable console output, researchers often need to programmatically access the raw mathematical objects for post-estimation testing, custom plotting, or exporting to specific pipeline formats.

Once the `.fit()` method successfully concludes, all underlying matrices, arrays, and scalars are permanently stored in the `model.results` dictionary.

```
        # Example: Extracting the robust standard errors and the threshold array
        raw_se = res.results['se']
        optimal_gammas = res.results['thresholds']

```

Dictionary Keys Reference:

`'thresholds'` *(1D Array)* The sorted point estimates of the identified thresholds ($\hat{\gamma}$).

`'ci_results'` *(List of Tuples)* The lower and upper bounds of the asymptotic confidence intervals for each threshold.

`'boot_results'` *(List of Dicts)* The sequential Hansen test diagnostics, containing `F_stat`, `p_value`, and critical values (`crit10`, `crit5`, `crit1`) for every step.

`'beta'` *(2D Column Vector)* The estimated slope coefficients ($\hat{\beta}$) for all regime-independent and regime-dependent variables.

`'se'` *(2D Column Vector)* The standard errors corresponding to $\hat{\beta}$. These are cluster-robust if `robust=True`.

`'t_stats'` & `'p_values'` *(2D Column Vectors)* The t-statistics and two-tailed p-values for individual coefficient significance.

`'ci_l'` & `'ci_u'` *(1D Arrays)* Lower and upper bounds of the confidence intervals for the slope coefficients.

`'_cons'` & `'se_cons'` *(Scalars)* The recovered model intercept and its standard error.

`'r2_w'`, `'r2_b'`, `'r2_o'` *(Scalars)* The Within, Between, and Overall R^2 goodness-of-fit metrics.

`'sigma_u'`, `'sigma_e'`, `'rho'` *(Scalars)* Variance components decomposing the individual fixed effects and the idiosyncratic error term.

`'F_overall'` & `'F_pval'` *(Scalars)* The global F-test statistic (or Wald test if `robust=True`) and its corresponding p-value for the joint significance of the model.

`'df_resid'` & `'df_tests'` *(Integers)* Degrees of freedom used for variance scaling and distribution testing, respectively.

`'ssrs'` *(1D Array)* The minimum Sum of Squared Residuals obtained for each sequential threshold added to the model.

`'lr_sequences'` *(List of 1D Arrays)* The raw evaluations of the $LR_1(\gamma)$ statistic across the search grid, required for custom plotting.

30 Troubleshooting and Common Errors

Empirical panel data is rarely perfectly well-behaved. The `pyxthreg` engine incorporates several internal checks and raises specific errors when the mathematical assumptions of the model are violated. Below are the most common computational issues and their resolutions:

- **`LinAlgError: Singular matrix`**
  Cause: This occurs during the Concentrated Least Squares estimation if your design matrix X suffers from perfect multicollinearity. This frequently happens if the user includes a regime-independent control variable that is perfectly correlated with the fixed effects, or if dummy variables fall into the "dummy variable trap."
  Solution: Review your `indep` variables. If you are using `time_fe=True`, do not manually include year dummies in the `indep` list, as the package automatically mathematically purges temporal effects.

- **`ValueError: Regime size falls below trimming percentage`**
  Cause: The algorithm attempted to evaluate a threshold γ that left fewer observations in a regime than dictated by the `trim` parameter (default 5%).
  Solution: If you have a relatively small dataset or if the structural break occurs at the extreme tail of the distribution, try lowering the parameter to `trim=0.01`.

- **`Warning: Bootstrap p-value = 1.000`**
  Cause: If the test yields exactly 1.0, the restricted linear model and the unrestricted threshold model likely produced the exact same Sum of Squared Residuals (SSR).
  Solution: This typically means the regime-dependent variable (`rx`) contains zero variance, or the threshold variable (`qx`) is a constant. Ensure your variables contain sufficient continuous variation.

31 References

- Davies, R. B. (1987). Hypothesis testing when a nuisance parameter is present only under the alternative. Biometrika, 74(1), 33-43.
- Hansen, B. E. (1999). Threshold effects in non-dynamic panels: Estimation, testing, and inference. Journal of econometrics, 93(2), 345-368.
- Hansen, B. E. (2000). Sample splitting and threshold estimation. Econometrica, 68(3), 575-603.
- Wang, Q. (2015). Fixed-effect panel threshold model using Stata. The stata journal, 15(1), 121-134.